Deep Learning

Tutorial 13: Denoising Auto Encoders

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # ***000000494171***

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

Implement the deep auto encoder
Implement the variational auto encoder

Implementation of these tasks are in *PyTorch*:

Denoising and Variational Autoencoders on MNIST

1. Introduction

Autoencoders are a class of neural networks designed to learn efficient representations of data by compressing input information into a lower-dimensional latent space and then reconstructing it. In this project, we explore two types of autoencoders: the denoising autoencoder (DAE) and the variational autoencoder (VAE). The denoising autoencoder is trained to reconstruct clean images from noisy inputs, effectively learning to remove noise while preserving the essential structure of the image. The variational autoencoder, on the other hand, is a probabilistic model that encodes inputs into a latent space characterized by a distribution, enabling both reconstruction and the generation of new data. The goal of this study is to implement both models on the MNIST dataset and analyze their performance.

2. Dataset

The MNIST dataset consists of 60,000 training images and 10,000 test images of handwritten digits, each sized 28 by 28 pixels in grayscale. Before feeding the data into the models, the images were normalized to a range of 0 to 1 to improve training stability and were reshaped to include a channel dimension to be compatible with convolutional layers. For training the denoising autoencoder, random Gaussian noise with a factor of 0.5 was added to simulate corrupted images. The noisy inputs allow the DAE to learn how to recover the original clean images, while the VAE is trained on the original images to learn a probabilistic latent representation.

3. Denoising Autoencoder (DAE)

The denoising autoencoder consists of an encoder and a decoder built using convolutional layers. The encoder compresses the noisy input image into a smaller latent representation, capturing its essential features, while the decoder reconstructs the image from this compressed form using transposed convolutional layers. The model is trained using binary cross-entropy loss, which measures the difference between the original clean images and the reconstructed outputs. After training, the DAE effectively removes noise from the images while preserving the shapes of the digits, demonstrating its ability to learn robust feature representations for denoising tasks.

4. Variational Autoencoder (VAE)

The variational autoencoder uses an encoder that maps input images into a latent space represented by a mean vector and a log-variance vector. A sampling layer generates a latent vector from this distribution, which is then passed to a decoder to reconstruct the input image. The VAE is trained using a combination of reconstruction loss, which ensures the output resembles the input, and Kullback-Leibler divergence, which regularizes the latent space to follow a standard normal distribution. This approach enables the VAE to learn a continuous, structured latent space that allows for the generation of new samples. While the reconstructed images are slightly smoother than the originals, the model successfully captures the underlying data distribution, demonstrating the generative capability of the VAE.

5. Results

The denoising autoencoder successfully removed noise from corrupted MNIST images, producing clear reconstructions that closely resembled the original digits. In contrast, the variational autoencoder reconstructed the digits with a slightly smoother appearance due to its probabilistic nature, but it also provided the ability to sample from the latent space and generate new, realistic digit images. These results illustrate the different strengths of the two models: the DAE excels at image restoration, while the VAE is more suited for generative modeling and exploring latent representations.

6. Conclusion

In summary, both the denoising autoencoder and the variational autoencoder demonstrate the power of autoencoders for representation learning and image processing. The DAE is particularly effective for noise removal and image enhancement, whereas the VAE provides a probabilistic framework that enables reconstruction as well as the generation of new data from a learned latent space. Understanding the differences between these models is important for selecting the appropriate approach depending on whether the goal is denoising or generative modeling. This project highlights the versatility and practical applications of autoencoders in computer vision tasks.

7.Full code

```
# ----------------------------

# Import Libraries

# ----------------------------

import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras import layers, Model

from tensorflow.keras.datasets import mnist


# ----------------------------

# Load and Preprocess Data

# ----------------------------

(x_train, _), (x_test, _) = mnist.load_data()
```

```
x_train = x_train.astype('float32') / 255.

x_test = x_test.astype('float32') / 255.


# Add channel dimension

x_train = np.expand_dims(x_train, -1)

x_test = np.expand_dims(x_test, -1)


# Add random noise for Denoising Autoencoder

noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=x_train.shape)

x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)


x_train_noisy = np.clip(x_train_noisy, 0., 1.)

x_test_noisy = np.clip(x_test_noisy, 0., 1.)


# ----------------------------

# Deep Denoising Autoencoder

# ----------------------------

input_img = layers.Input(shape=(28,28,1))


# Encoder
```

```
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = layers.Conv2D(64, (3,3), activation='relu', padding='same', strides=(2,2))(x)
x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
encoded = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)

# Decoder
x = layers.Conv2DTranspose(64, (3,3), strides=(2,2), activation='relu', padding='same')(encoded)
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
decoded = layers.Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)

autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.summary()

# Train DAE
autoencoder.fit(x_train_noisy, x_train,
                epochs=10,
                batch_size=128,
                shuffle=True,
                validation_data=(x_test_noisy, x_test))

# Visualize DAE Results
```

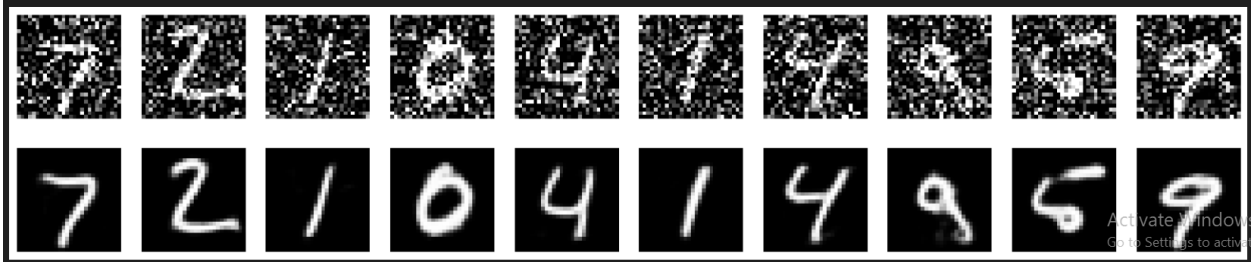
```
decoded_imgs = autoencoder.predict(x_test_noisy)
n = 10
plt.figure(figsize=(20,4))
for i in range(n):
    ax = plt.subplot(2, n, i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
    plt.axis('off')
    ax = plt.subplot(2, n, i+1+n)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.axis('off')
plt.show()


# ----------------------------
# Variational Autoencoder (VAE)
# ----------------------------
latent_dim = 2


# Sampling Layer
class Sampling(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        epsilon = tf.random.normal(shape=tf.shape(z_mean))
```

```
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon


# Encoder
encoder_inputs = layers.Input(shape=(28,28,1))
x = layers.Flatten()(encoder_inputs)
x = layers.Dense(256, activation='relu')(x)
z_mean = layers.Dense(latent_dim)(x)
z_log_var = layers.Dense(latent_dim)(x)
z = Sampling()([z_mean, z_log_var])
encoder = Model(encoder_inputs, [z_mean, z_log_var, z], name="encoder")


# Decoder
latent_inputs = layers.Input(shape=(latent_dim,))
x = layers.Dense(256, activation='relu')(latent_inputs)
x = layers.Dense(28*28, activation='sigmoid')(x)
decoder_outputs = layers.Reshape((28,28,1))(x)
decoder = Model(latent_inputs, decoder_outputs, name="decoder")


# VAE Model
class VAE(Model):
    def __init__(self, encoder, decoder, **kwargs):
        super(VAE, self).__init__(**kwargs)
```

```
        self.encoder = encoder
        self.decoder = decoder

    def call(self, inputs):
        z_mean, z_log_var, z = self.encoder(inputs)
        reconstructed = self.decoder(z)

        # Compute losses
        reconstruction_loss = tf.reduce_mean(
            tf.reduce_sum(tf.keras.losses.binary_crossentropy(inputs, reconstructed), axis=(1,2,3))
        )
        kl_loss = -0.5 * tf.reduce_mean(
            tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=1)
        )
        self.add_loss(reconstruction_loss + kl_loss)
        return reconstructed

vae = VAE(encoder, decoder)
vae.compile(optimizer='adam')
vae.summary()

# Train VAE
```

```
vae.fit(x_train, x_train,
        epochs=10,
        batch_size=128,
        validation_data=(x_test, x_test))


# Visualize VAE Results
decoded_imgs_vae = vae.predict(x_test[:10])
n = 10
plt.figure(figsize=(20,4))
for i in range(n):
    ax = plt.subplot(1, n, i+1)
    plt.imshow(decoded_imgs_vae[i].reshape(28,28), cmap='gray')
    plt.axis('off')
plt.show()
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 12s 20ms/step - loss: 0.2200 - val_loss: 0.1005
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.1005 - val_loss: 0.0970
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0975 - val_loss: 0.0954
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0955 - val_loss: 0.0957
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0943 - val_loss: 0.0935
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0938 - val_loss: 0.0935
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0933 - val_loss: 0.0924
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0928 - val_loss: 0.0920
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0923 - val_loss: 0.0917
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 14ms/step - loss: 0.0919 - val_loss: 0.0915
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```



```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - loss: 253.4645 - val_loss: 173.9438
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 172.7242 - val_loss: 168.9868
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 168.1934 - val_loss: 166.1380
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 165.1552 - val_loss: 163.8001
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 162.4134 - val_loss: 162.0075
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 161.4977 - val_loss: 160.6533
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 159.9570 - val_loss: 159.5172
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 158.7105 - val_loss: 158.6012
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 158.1935 - val_loss: 158.0443
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 157.1960 - val_loss: 157.2391
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 715ms/step
```